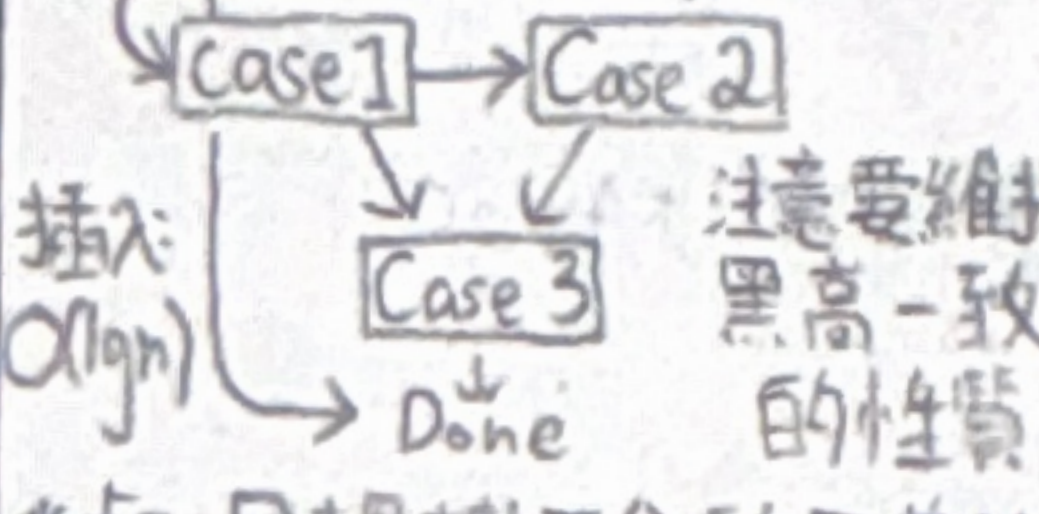


Chapter.13 Red-Black Trees 紅黑樹 !!重要!! #DS

- 性質:
- 1. Node Color 非黑即紅
 - 2. Root 必黑
 - 3. 葉子(NIL)必黑
 - 4. 紅-紅 Node 不相連
 - 5. 黑高怎麼走都要一致

插入 Insertion (同BS Tree)

和二元搜尋樹插法一樣, 小左大右, 但 Color 先塗紅, 後 RB-Insert-Fixup(T,z) 修正



*叔: 同棵樹下爸的兄弟!!

- Case 1. 叔紅的
- 父: 叔黑, 爺紅, 上兩層
- Case 2. 叔黑, 我在右
- 父左轉, 我到左子!
- Case 3. 叔黑, 我在左
- 父黑, 爺紅, 爺右轉, 完成!

- Case 1. 叔紅的
- 父: 叔黑, 爺紅, 上兩層
 - 和父在左子的情況相同
- Case 2. 叔黑, 我在左
- 父右轉, 我到右子
- Case 3. 叔黑, 我在右
- 父黑, 爺紅, 爺左轉, 完成!

刪除 DELETION

*刪除: O(lgn) (刪紅沒關係, 刪黑要調整)

- Case 1. 沒兒子
- 直接刪
- Case 2. 有一兒
- 把自己的兒子連到父親
- Case 3. 有2兒
- 找 Successor
 - 把 Successor 複製過來
 - 刪 Successor!

RB Tree Function Pseudocode Call

- LeftRotate(T,x) / RightRotate(T,x)
- RB-Insert(T,z) 插入
- RB-Insert-Fixup(T,z) 插入修正
- RB-Delete(T,z) 刪除
- RB-Delete-Fixup(T,x) 刪除修正

Chapter.12 Binary Search Tree 二元搜尋樹 #DS

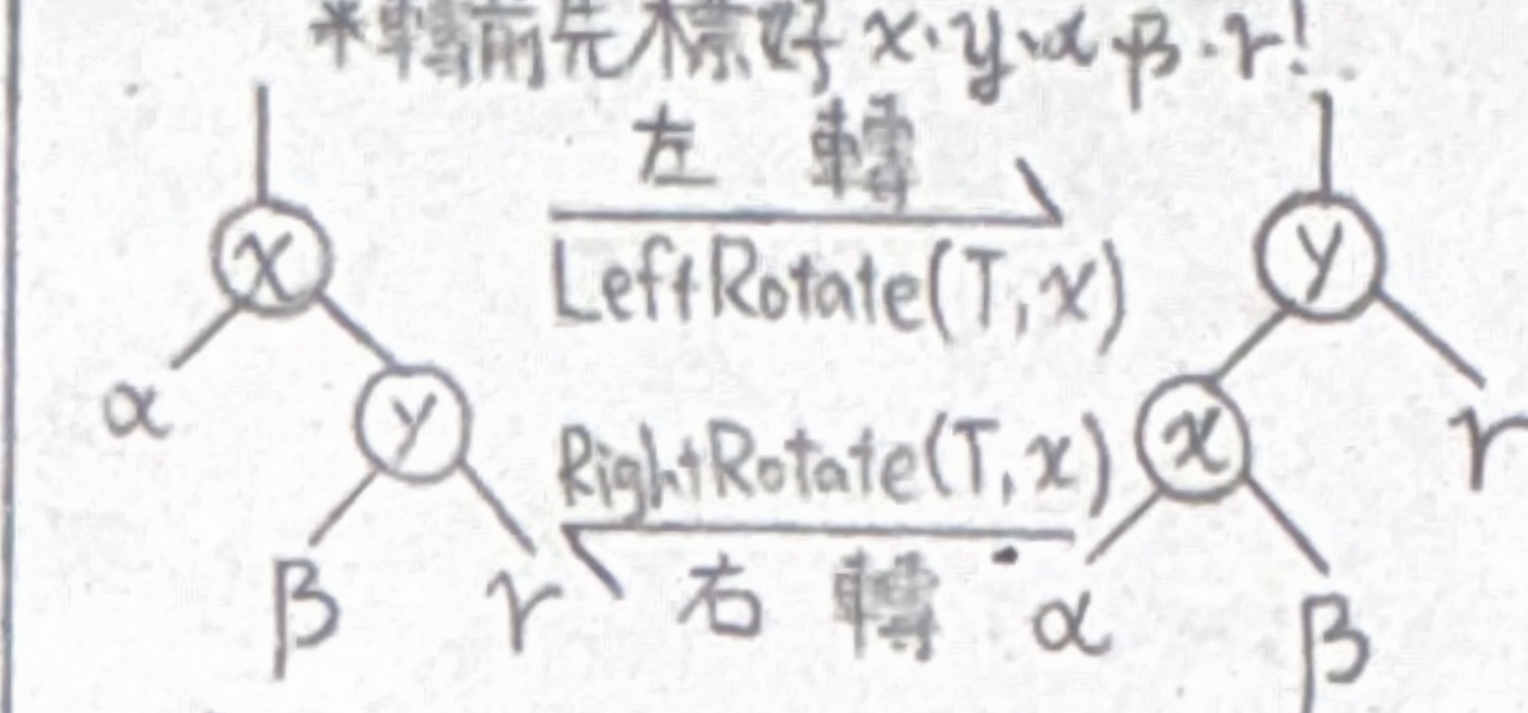
性質: 左子樹 < 我 / 右子樹 > 我

操作: Successor(下一個自己的) / Predecessor(前一個自己的) / Search / Min / Max → O(lgn)

Insert O(lgn):

TreeInsert(T,z):

```
y ← Nil, x ← T.root
while x ≠ Nil
  y ← x
  if z.key < x.key
    x ← x.left
  else x ← x.right
z.parent ← y
if y = Nil: T.root ← z
else if z.key < y.key: y.left ← z
else y.right ← z
```



不論怎麼轉, 樹的性質和大小關係都有被維持。 $\alpha \leq x \leq \beta \leq \gamma \leq r$ (btree性質)

Left Rotate 左轉

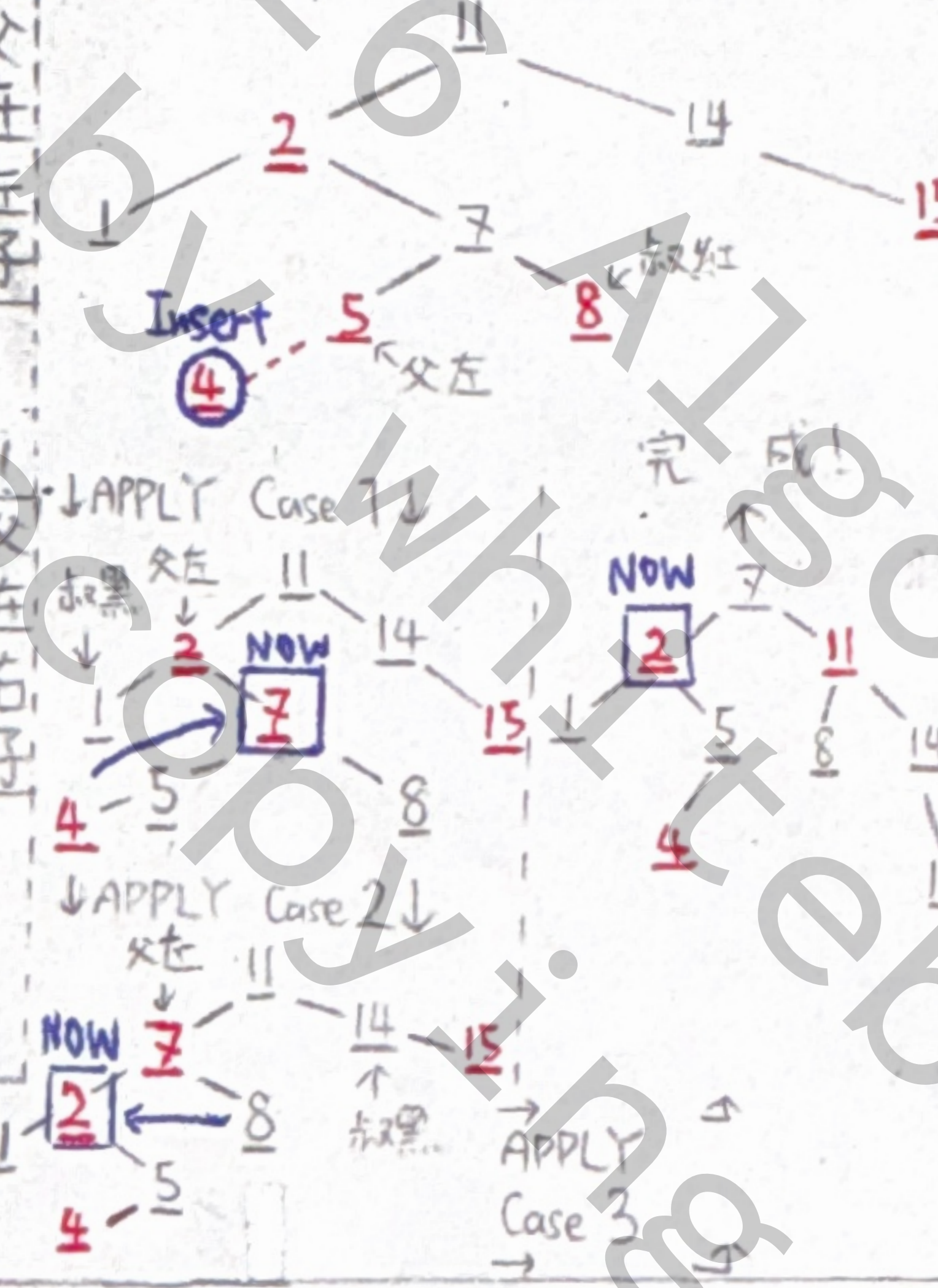
- ① $x \rightarrow \beta, y \rightarrow x$
- ② x 的 Parent $\rightarrow y$
- ③ $y \rightarrow x, x \rightarrow \gamma$

Right Rotate 右轉

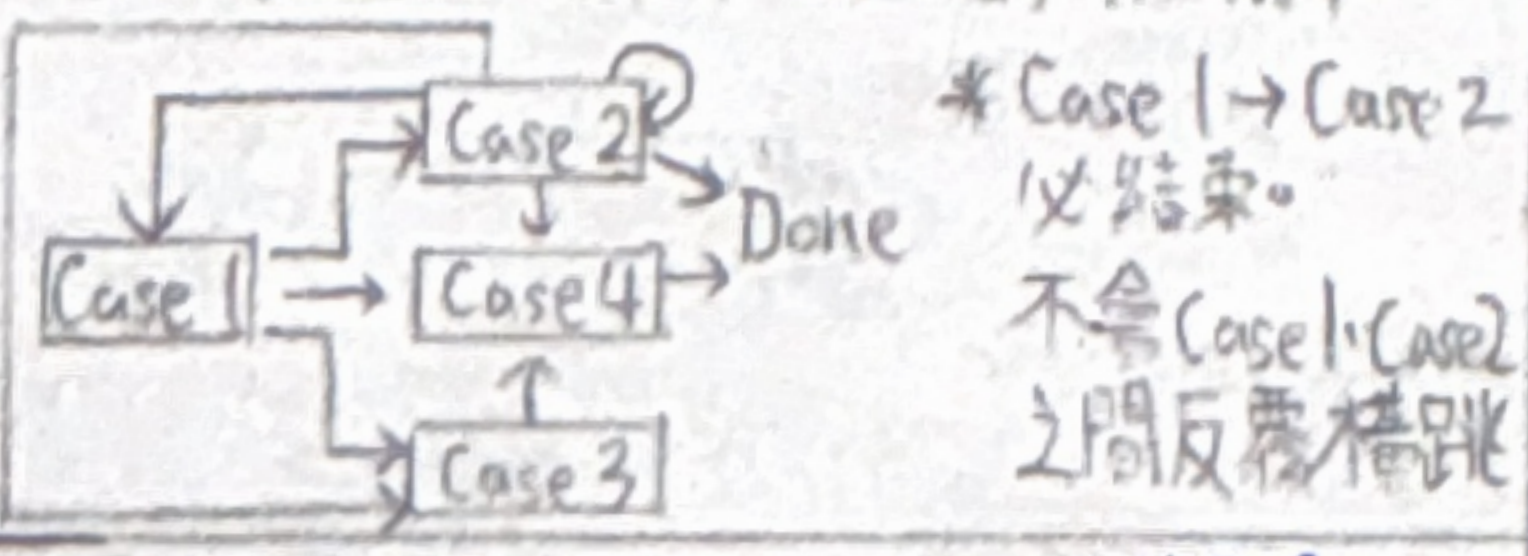
- ① $y \rightarrow \beta, x \rightarrow \beta$
- ② y 的 Parent $\rightarrow x$
- ③ $x \rightarrow \gamma, y \rightarrow x$

*root 的 Parent 是 Nil!! 旋轉: O(1)

Insert Sample (Consider Parent is Left Only)



- Case 1. 兄是紅
- 兄黑, 父紅
- Case 2. 兄黑, 兄2兒黑
- 我兄黑, 父黑, 父左轉
- Case 3. 兄黑, 兄左兒紅, 右兒黑
- 兄右轉, 到 Case 4
- Case 4. 兄黑, 兄右兒紅
- 我兄黑, 父黑, 父左轉



Deletion O(lgn): 刪除分成3種Case (同紅黑樹)

TreeDelete(T,z)

```
if z.left = Nil or z.right = Nil
  y ← z
else
  y ← TreeSuccessor(z)
if y.left ≠ Nil
  x ← y.left
else x ← y.right
if x ≠ Nil
  x.parent ← y.parent
if y.parent = Nil:
  T.root ← x
else if y = y.parent.left:
  y.parent.left ← x
else y.parent.right ← x
if y ≠ z:
  copy all y's data to z
return y
```

Chapter.11 Hash Tables 哈希表

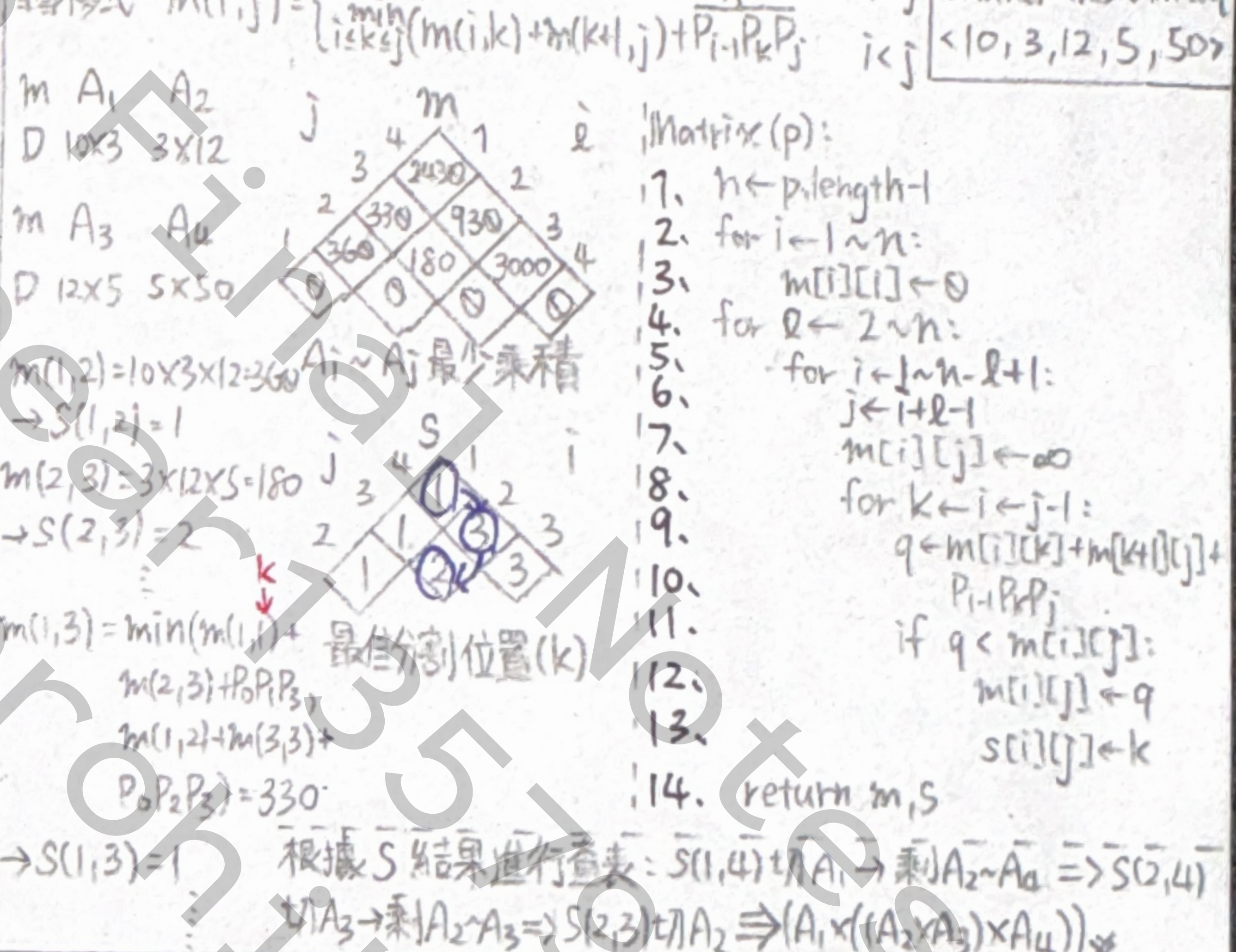
Linear Probing → 問隔壁!

Quadratic Probing → 構造2次方程

Double Hashing → 用2個hash func.

Chapter.15 Dynamic Programming 動態規劃 !!重要!! #ALGO

Matrix Chain Multiplication O(n^3)



Optimal Polygon Triangulation O(n^3)

轉移式: $t(i,j) = \min_{i \leq k \leq j-1} (t(i,k) + t(k,j) + w(v_i, v_k, v_j))$

和 Matrix Chain 數學上-一致!

參數 $P \Rightarrow v, w, n$

Line 1 Delete

Line 9, 10 替換轉移

SAMPLE: $n=4$ (4頂點)

- $w(v_0, v_1, v_2) = 10$
- $w(v_1, v_2, v_3) = 12$
- $w(v_0, v_1, v_3) = 11$
- $w(v_1, v_2, v_3) = 15$

Longest Common Subsequence

$O(nm)$, $LCS(X, Y)$:

轉移式 $C(i, j)$:

$$C(i, j) = \begin{cases} 0 & i=0 \text{ or } j=0 \\ C(i-1, j-1) + 1 & X_i = Y_j \\ \max(C(i-1, j), C(i, j-1)) & X_i \neq Y_j \end{cases}$$

SAMPLE: $X = ABC, Y = BDC$

轉移前, $i=0, j=0$ 填 0

		j	0	1	2	3
i	0	0	0	0	0	0
1	A	0	0	1	1	1
2	B	0	1	1	2	2
3	C	0	1	1	2	2

Answer: BC

Case 1: $X_i = Y_j$
Case 2, 3: $X_i \neq Y_j$

上左取大

Optimal BST

$O(n^3)$

轉移式: $e(i, j)$:

$$e(i, j) = \min_{i \leq k \leq j} \{e(i, k-1) + e(k, j) + w(i, j)\}$$

子樹內機率: $w(i, j) = w(i, j-1) + p[j] + q[j]$

最小期望搜尋成本

SAMPLE: $n=3$
 $P: 1, 2, 3, 1$
 $Q: 1, 1, 1, 1$

create 3 array: e, s, root

for $j=1 \sim n+1$:

for $i=1 \sim j-1$:

for $k=i+1 \sim j$:

if $e(i, k-1) + e(k, j) + w(i, j) < e(i, k)$:

$e(i, j) = e(i, k-1) + e(k, j) + w(i, j)$

$s[j] = k$

return e, root

Rod Cutting

$O(n^2)$

原Cutting Rod 轉移式: $r(n) = \max_{1 \leq i \leq n} (P_i + r(n-i))$

考慮「切」的成本的式: $r(n) = \max_{1 \leq i \leq n} (P_i + r(n-i) - C)$

SAMPLE: $C=2$

i	1	2	3	4	5	6	7	8	9	10
P	1	3	6	6	10	9	8	13	8	12

普通Cutting Rod:

1 CutRod(P, n):

2 create 2 array r, s

3 $r[0] \leftarrow 0$

4 for $j=1 \sim n$:

5 now $\leftarrow -\infty$

6 for $i=1 \sim j$:

7 if $now < P[i] + r[j-i]$:

8 now $\leftarrow P[i] + r[j-i]$

9 $s[j] \leftarrow i$

10 $r[j] \leftarrow now$

11 return r, s

考慮「切」有成本的Cutting Rod

Line 1 $\rightarrow$ CutRod(P, n, C)

Line 3 $\rightarrow$ now $\leftarrow P[i] + r[j-i] - C$

Line 7 $\rightarrow$ $P[i] + r[j-i] - C$

01 Knapsack Problem

$O(nC)$

轉移式 $dp(i, j)$:

$$dp(i, j) = \begin{cases} 0 & i=0 \text{ or } j=0 \\ dp(i-1, j) & w_i > j \\ \max(dp(i-1, j), dp(i-1, j-w_i) + v_i) & w_i \leq j \end{cases}$$

SAMPLE: $C=5, (w_1, v_1)=(2, 3), (w_2, v_2)=(3, 4), (w_3, v_3)=(4, 5)$ Knap(V, w, n, C):

create an array $dp[0 \sim n][0 \sim C]$

for $j=0 \sim C$: $dp[0][j] \leftarrow 0$

for $i=1 \sim n$:

for $j=1 \sim C$:

if $w_i > j$: $dp[i][j] \leftarrow dp[i-1][j]$

else: $dp[i][j] \leftarrow \max(dp[i-1][j], dp[i-1][j-w_i] + v_i)$

return $dp[n][C]$

Chapter 9 Medians and Order Statistics

#ALGO

SELECT(A, i) $\rightarrow O(n)$ 時間內回傳 A 中第 i 小的元素

$\rightarrow$ 保證每次分割都有好比例, 不會退化到 $O(n^2)$ (Randomized-Select)

1. n 個元素, 每 5 個分一組

2. 用插入排序找出每組中位數

3. 找出這 5 個中位數的中位數

4. 以 x 為基準分割陣列

5. 判斷目標落點, 遞迴尋找

Proof:

5 個元素一組, 共 $\lceil n/5 \rceil$ 組

x 是中位數的中位數, 所以至少一半組的中位數 $\geq x$, 每組至少有 3 個元素 $\geq x$ (包括含 x 的最後一組, 分不到 5 個元素):

$3(\lceil n/5 \rceil - 2) \geq 3n/5 - 6$ 個元素 $\geq x$

每層遞迴最多: $\frac{7n}{10} + 6$ 個元素

$T(n) \leq T(\lceil n/5 \rceil) + T(\frac{7n}{10} + 6) + O(n)$

$\leq C(\frac{n}{5}) + C(\frac{7n}{10} + 6) + an$

$\leq \frac{9n}{10} + 7c + an$ 算這串 ≤ 0 , 求 n

$= Cn + (-\frac{9c}{10} + 7c + an)$ 得 $C \geq 20a$

$\Rightarrow T(n) = O(n)$

Chapter 8 Sorting in Linear Time

#ALGO

Counting Sort $O(n+k)$

	1	2	3	4	5
A	4	1	3	4	3

Radix Sort $O(d(n+k))$

從個位數往後排

326 751 608

453 $\rightarrow$ 453 $\rightarrow$ 326

608 $\rightarrow$ 326 $\rightarrow$ 751

751 608 453

326 751 608

453 $\rightarrow$ 完成!

Bucket Sort $O(n+k)$

假設所有 key 都在 $0 \sim 1$ 間隨機分布 $\rightarrow$ 建 k 個 Bucket, Sort

A: 0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12

B: 0.12 $\rightarrow$ 0.17, 0.21 $\rightarrow$ 0.26, 0.39 $\rightarrow$ 0.72 $\rightarrow$ 0.78, 0.94

Chapter 16 Greedy Algorithms

貪婪演算法 #ALGO

ActivitySelect(e): $O(n \log n)$

sort e by f_i , 由短到快

create an array A

A.push-back(e[1])

$i \leftarrow 1$

for $m=2 \sim n$:

if $e[m].s \geq e[i].f$:

A.push-back(e[m])

$i \leftarrow m$

return A

如果帶價值的活動或工作要 DP 解! (01 背包問題)

Proof in BST if x have 2 子, 其 Successor 沒左子

Let x is a BST nodes have 2 Child.

$\therefore x$ 有一棵右子樹

According BST 性質, 所有 $> x$ 的 node 都在 x 的右子。

Assume x's successor y have 左子 z

據 BST 性質, 若 z 是 y 的左子, 則 $z < y$

而 z 又在 x 的右子樹, 表 $x < z$

綜上所述, 可得關係式 $x < z < y$ 若存在 1 node z 介於 x 和 y 間, 那 z 應比 y 更接近 x, y 便不會是 x 的 Successor。

$\rightarrow$ 與假設產生矛盾, $\therefore$ 假設不成立

$\rightarrow$ 若 node 有子, 其前驅必不存在左子。